O que documentar

O que documentar

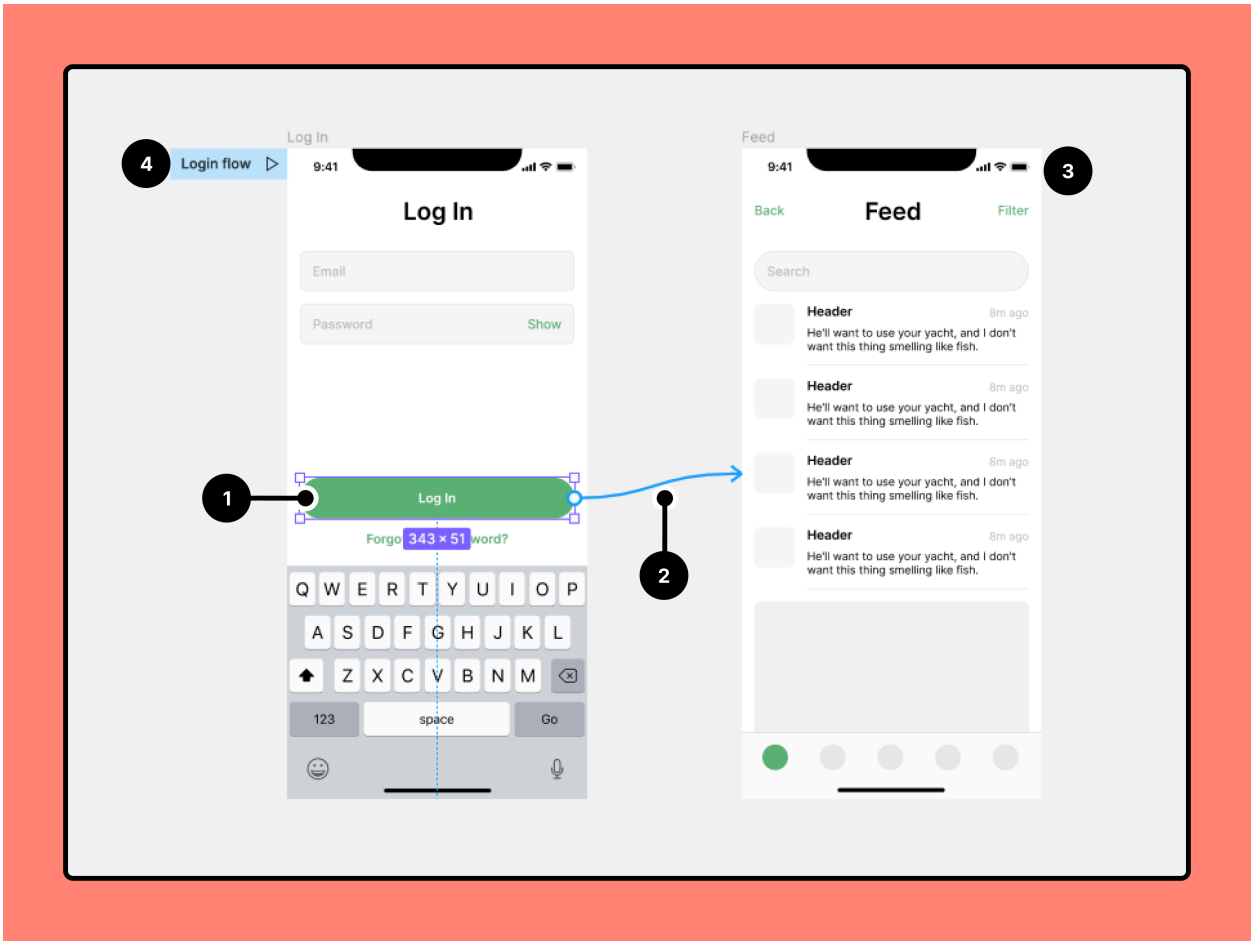
Você chegou até aqui sabendo o que são regras de negócio, requisitos, histórias de usuário e critérios de aceite. O vocabulário está claro. Mas agora vem a pergunta que aparece toda vez que alguém senta para especificar de verdade:

Quanto eu preciso escrever?

Escrever de menos gera ambiguidade, dando margem para implementações diferentes porque "achei que era assim". Escrever demais gera documentação que ninguém lê, fica desatualizada em dois sprints e dá a falsa sensação de que o projeto está protegido quando não está.

O equilíbrio tem nome na literatura ágil: *just enough* documentation. A mínima viável que permite ao time construir o produto certo de forma eficiente.

O que as telas já documentam por você



A tela cobre o "como parece". Layout, hierarquia visual, componentes, fluxo de navegação. Redocumentar o que já está visível é desperdício.

O que a tela não cobre é o "como se comporta". Comportamentos condicionais, estados do sistema, fluxos de erro, regras de negócio por trás de algo que parece visual, permissões que mudam o que o usuário vê. Tudo isso é invisível num protótipo estático. É exatamente isso que precisa ser escrito.

Como saber o quanto documentar

Passo 1 — Conheça quem vai ler antes de escrever

O nível de detalhe necessário depende diretamente de quem vai construir. Antes de especificar com um dev novo, vale uma pergunta direta: "Quando você recebe uma história com uma lacuna, o que você faz?"

Você vai encontrar basicamente três perfis:

- O **inferidor** preenche as lacunas sozinho. Documente apenas o que ele genuinamente não teria como adivinhar.
- O **perguntador** para e te chama antes de implementar. Antecipe as dúvidas prováveis e responda antes que elas apareçam.
- O **literal** implementa exatamente o que está escrito. Se não está escrito, não existe. Para ele, implícito não é suficiente.

Você não vai mudar o perfil do desenvolvedor. Você vai adaptar sua documentação a ele.

Passo 2 — Pergunte o que a tela não responde

Para cada tela ou componente, aplique a pergunta gatilho: *o que está acontecendo aqui que um desenvolvedor não consegue ver?*

Categoria	Pergunta prática	Exemplo do que documentar
Estados do sistema	Essa tela tem mais de uma versão dependendo de uma condição?	"Botão desabilitado enquanto o campo estiver vazio"
Erros e exceções	O que acontece quando algo dá errado?	"Se a API falhar, exibir mensagem X e manter os dados preenchidos"
Regras de negócio	Há uma lógica por trás de algo que parece visual?	"Só usuários com perfil Admin veem esse campo"
Fluxos alternativos	Existe um caminho que o usuário pode tomar que não está prototipado?	"Se o usuário já tiver conta, pular etapa 2"
Dependências de sistema	Esse comportamento depende de outro sistema, API ou serviço?	"O status é atualizado em tempo real via webhook"
Timing e assincronismo	Tem carregamento, delay, polling ou atualização automática?	"Lista recarrega a cada 30 segundos automaticamente"
Permissões e contexto	O comportamento muda dependendo de quem está usando?	"Usuário free não consegue clicar, vê tooltip de upgrade"

Não pense no que você sabe. Pense no que está invisível. Se a tela já responde claramente, não precisa de texto. Se exige conhecimento de negócio, estado do

sistema ou regra que não aparece na interface, precisa. Uma ferramenta que pode ajudar bastante nisso é o checklist DIREC, disponível na biblioteca de técnicas da DesignWiki.

Passo 3 — Aplique o teste do "consegue testar?"

Para cada item que levantou no passo anterior, pergunte: *o QA consegue verificar se isso foi implementado corretamente só olhando o Figma?*

Se sim, apaga. Se a resposta exigir uma frase de comportamento — "quando X acontece, o sistema faz Y" — mantém.

Passo 4 — Escreva comportamento, não intenção

Quando algo precisa ser documentado, a forma importa. Não escreva o que você quer, escreva o que o sistema faz. A estrutura que funciona é simples: **quando [condição], o sistema [faz X]**.

❌ Evitar	✅ Preferir
"O botão deve ser intuitivo"	"O botão só fica ativo quando todos os campos obrigatórios estiverem preenchidos"
"Tratar erro de forma amigável"	"Se a requisição falhar, exibir: 'Não foi possível salvar. Tente novamente.'"
"Respeitar hierarquia de usuários"	"Usuários com perfil Viewer não veem o botão Editar"

Passo 5 — Calibre com cada entrega

O nível certo não é fixo. Se algo foi implementado diferente do esperado, você documentou pouco ou a frase ficou ambígua. Se o dev perguntou algo que estava escrito, a frase não estava clara o suficiente. Se perguntou algo que não estava, adicione essa categoria ao seu radar.

Com três ou quatro histórias entregues, você já tem o nível calibrado para aquela pessoa. Quando o time mudar, você começa de novo, mas com muito mais consciência do processo.

Saber o que documentar resolve metade do problema. A outra metade é estruturar o processo de especificação do começo ao fim, de forma que nenhuma decisão importante fique pra trás. É exatamente isso que o Framework CERVE propõe, e é o que veremos a seguir.